Programación de funciones en Python

Las funciones representan uno de los pilares fundamentales de la programación estructurada y funcional. Permiten encapsular lógica, modularizar programas, mejorar la legibilidad del código y favorecer la reutilización.

1 FUNCIONES EN PYTHON

1.1 DEFINICIÓN DE FUNCIONES EN PYTHON

Las funciones en Python se definen mediante la palabra clave `def`. Su sintaxis básica es:

```
def nombre_de_la_funcion(parametros):
    """
    Documentacion de la funcion
    """
    # Cuerpo de la funcion
    return valor
```

PARÁMETROS Y ARGUMENTOS

1. **Parámetros posicionales**
2. **Parámetros por defecto**
3. ***args** para parámetros variables posicionales.
4. ****kwargs** para parámetros variables nombrados.

```
def ejemplo(a, b=10, *args, **kwargs):
    print(f"a es un parámetro posicional con valor: {a}")
    print(f"b es un parámetro por defecto con valor: {b}")
    print(f"args es un parámetro de variables posicionales con valor:
    ↪ {args}")
    print(f"kwargs es un parámetro de variables nombrados con valor:
    ↪ {kwargs}")
```

```
ejemplo(
    5, # a: parámetro posicional obligatorio
    20, # b: parámetro con valor por defecto, pero aquí sobrescrito
```

```
    1, 2, 3, # *args: argumentos posicionales adicionales
    nombre="Gustave", # **kwargs: argumentos nombrados adicionales
    edad=32
)
```

```
a es un parámetro posicional con valor: 5
b es un parámetro por defecto con valor: 20
args es un parámetro de variables posicionales con valor: (1, 2, 3)
kwargs es un parámetro de variables nombrados con valor: {'nombre':
↪  'Gustave', 'edad': 32}
```

SCOPE: ÁMBITO DE VARIABLES

Python utiliza el modelo **LEGB** (Local, Enclosing, Global, Built-in) para buscar variables.

1. **Local** Son las variables definidas dentro de una función, o dentro de un bloque que crea su propio espacio local (como una lambda).

```
def f():
    x = 10
    print(x)
```

2. **Enclosing** Son las variables del ámbito externo cuando una función está anidada dentro de otra.

```
def exterior():
    x = 20
    def interior():
        print(x)
    interior()
```

3. **Enclosing** Son las variables del ámbito externo cuando una función está anidada dentro de otra.

```
def exterior():
    x = 20
    def interior():
        print(x)
    interior()
```

4. **Global** Variables definidas en el módulo actual, es decir, fuera de cualquier función.

```python
x = 30

def f():
    print(x)
```

5. **Global** Variables y funciones incluidas en Python por defecto, como `len` o `Exception`.

```python
def f():
    print(len([1, 2, 3]))
```

EJEMPLO COMPLETO DE FUNCIONAMIENTO DE LA LÓGICA LEGB

```python
x = "global"

def exterior():
    x = "enclosing"

    def interior():
        x = "local"
        print(x)

    interior()
    print(x)

exterior()
print(x)
print(len("hey"))
```

FUNCIONES DE ORDEN SUPERIOR

Una función de orden superior es aquella que recibe una o más funciones como argumento y/o retorna una función.

```python
def aplicar(func, valor):
    return func(valor)

def cuadrado(x):
```

```
    return x**2

print(aplicar(cuadrado, 5))
```

2 DOCSTRING

Un docstring es una cadena de documentación que se coloca al inicio de módulos, funciones clases o métodos para describir qué hacen, cómo se usan y cualquier útil para quien lea o use el código.
Para definir un docstring se utilizan comillas triples como primera línea dentro de la función, módulo o clase:

```
def funcion():
    """
    Este es un docstring.
    Describe qué hace la función.
    """
    pass
```

Hay varios estilos aceptados:

- **reStructuredText (reST)**. El estándar documentación oficial de Python. No muy amigable.

```
def dividir(a, b):
    """
    Divide a entre b.

    :param a: Dividendo
    :type a: float
    :param b: Divisor
    :type b: float
    :returns: Resultado de la división
    :rtype: float
    :raises ZeroDivisionError: si b es cero
    """
    return a / b
```

- **Google docstrings** La forma de documentación recomendada por Google.

```python
def dividir(a, b):
    """
    Divide a entre b.

    Args:
        a (int | float): Número dividendo.
        b (int | float): Número divisor. No debe ser cero.


    Returns:
        float: Resultado de la división.

    Raises:
        ZeroDivisionError: Si b es igual a cero.
        TypeError: Si a o b no son numéricos.
    """
    return a / b
```

- **NumPy/SciPy docstrings** Combinación de reStructured y Google docstrings desarrollada por NumPy.

```python
def dividir(a, b):
    """
    Divide un número entre otro y retorna el resultado.

    Parameters
    ----------
    a : int or float
        Número dividendo.
    b : int or float
        Número divisor. No debe ser cero.

    Returns
    -------
    float
        Resultado de la división.
```

```
    Raises
    ------
    ZeroDivisionError
        Si `b` es igual a cero.
    TypeError
        Si `a` o `b` no son numéricos.
    """
    return a / b
```

3 FUNCIONES LAMBDA

Las funciones lambda son funciones anónimas, de una sola expresión, definidas con:

```
lambda argumentos: expresion
```

```
clasificar = lambda x: "Par" if x % 2 == 0 else "Impar"
print(f"El numero 10 es {clasificar(10)}")
```

Las principales características para usar una función lambda son:

- No requieren nombre.
- Solo pueden contener una expresión.
- Son útiles cuando la función se usa una vez.

Las lambdas permiten el uso de múltiples argumentos y no solo de uno dependiendo del objetivo que queramos cumplir con la lambda:

```
suma = lambda a, b: a + b
print(f"La suma de 1 y 3 es {suma(1, 3)}")
```

Una lambda permite recibir otra lambda como cuerpo:

```
potenciador = lambda n: (lambda x: x**n)
elevar_cuadrado = potenciador(2)
print(f"3 al cuadradado es {elevar_cuadrado(3)}")
```

!!

Cuando estés diseñando lambdas, intenta evitar los siguientes fallos:

- Intentar usar múltiples instrucciones en una lambda.
- Abusar de las lambdas en lugar de usar `def`.
- Usar lambdas muy complejas y difíciles de leer.
- Olvidar que las lambdas no pueden tener docstrings.

3.1 USO AVANZADO DE LAMBDAS

Una de las principales funcionalidades de las funciones lambda es ser pasadas como parámetro para otras funciones con un comportamiento más complejo, por ejemplo, pueden ser pasadas como parámetro con la función, `sorted`, `filter` o `map` para potenciar su funcionalidad.

MAP()

La función `map()` aplica uan función dada a cada elemento de un iterable y devuelve un iterador con los resultados, permitiendo transformar secuencias sin tener que utilizar bucles de forma explícita.

```
numeros = [1, 2, 3, 4]
resultado = list(map(lambda x: x*2, numeros))
print(resultado)
```

```
estudiantes = [
  {"nombre": "Ana", "notas": [7, 8, 9]},
  {"nombre": "Brando", "notas": [5, 6, 5]}
]

promedios = list(map(
  lambda e: (e["nombre"], sum(e["notas"])/len(e["notas"])),
  estudiantes
))

print(promedios)
```

FILTER()

La función `filter()` se usa para seleccionar elementos de un iterable que cumplan una condición específica, aplicando una función que devuelva `True` o `False` a cada elemento. `filter()` de

volverá como un iterador los elementos que al aplicar la función hayan devuelto True.

```
numeros = [1, 2, 3, 4]
pares = list(filter(lambda x: x % 2 == 0, numeros))
print(pares)
```

SORTED()

La función `sorted()` es una función que devuelve uan nueva lista con los elementos de un iterable ordenados.

```
personas = {
    "maria": {"edad": 23},
    "lope": {"edad": 22}
}

orden_edad = sorted(personas.items(), key=lambda x: x[1]["edad"])
print(orden_edad)
```

!! Recuerda que `items()` transforma un diccionario en una lista de tuplas conformada por la clave en la posición 0 de la tupla y el valor en la posición 1. Por lo tanto, en la lambda del ejemplo anterior, estamos ordenando tomando como clave de ordenación el valor del diccionario y de dicho valor, que en esta ocasión es también un diccionario, el campo edad.

4 RECURSIVIDAD

La recursividad es una técnica de programación en la que una función se llama a sí misma para resolver un problema dividiéndolo en partes más pequeñas.

!! Recursividad es cuando una función se define en términos de sí misma, con el objetivo de resolver un problema repitiendo una lógica hasta llegar a un caso base.

Toda función recursiva debe tener:

1. **Caso base**. Es la condición que detiene la recursión. Si no existe la recursión es infinita y causa un error `RecursionError`.

2. **Caso recursivo**. Parte de la función donde se llama a sí misma con un problema más pequeño.
3. **Progresión hacia el caso base**. Cada llamada debe avanzar hacia el final.

```python
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

4.1 ALGORITMOS DE ORDENACIÓN RECURSIVOS: QUICKSORT

Quick sort utiliza divide y vencerás, eligiendo un pivote y dividiendo la lista en elementos menores que el pivote, el pivote y elementos mayores que el pivote. Luego ordena recursivamente ambos lados.

```python
def quicksort(lista):
    if len(lista) <= 1:
        return lista

    pivote = lista[len(lista) // 2]

    menores = [x for x in lista if x < pivote]
    iguales = [x for x in lista if x == pivote]
    mayores = [x for x in lista if x > pivote]

    return quicksort(menos) + iguales + quick_sort(mayores)

print(quicksort([5, 3, -1, 2])
```

!! Existen videos que ejemplifican el funcionamiento de los algoritmos de ordenación a través de bailes tradicionales. Por ejemplo: el siguiente video muestra el funcionamiento del algoritmo QuickSort a través de una danza tradicional húngara.

4.2 PROBLEMA COMPLEJO: TORRES DE HANÓI

El problema de las Torres de Hanói es uno de los problemas clásicos más importantes en algoritmos y matemáticas recreativas. Es un ejemplo perfecto para comprender recursividad, división de problemas, subproblemas dependientes, y complejidad exponencial.

El problema fue descrito por primera vez en 1883 por el matemático francés Édouard Lucas. Existe una leyenda que habla de unos monjes encargados de resolver el problema con 64 discos; según la historia, cuando terminen, el mundo llegará a su fin. Resolver 64 discos requeriría $2^{64} - 1$ movimientos, lo cual es un número extremadamente grande.

Existen tres reglas fundamentales:

1. Solo se puede mover un disco a la vez.
2. Solo se puede mover el disco superior de cualquiera de las tres pilas.
3. No se puede colocar un disco grande sobre un disco más pequeño.

El objetivo es mover la pila completa desde el poste de origen hasta el poste de destino respetando las reglas anteriores.

El problema es un ejemplo clásico de recursividad. ara mover una torre de N discos, la estrategia consiste en:

- Mover los N-1 discos superiores desde el poste de origen al poste auxiliar.
- Mover el disco más grande (el disco N) desde el poste de origen al poste destino.
- Mover los N-1 discos que quedaron en el poste auxiliar hacia el poste destino.

```python
def hanoi(n, origen, destino, auxiliar):
    if n == 1:
        print(f"Mover disco 1 de {origen} a {destino}")
        return

    hanoi(n - 1, origen, auxiliar, destino)
    print(f"Mover disco {n} de {origen} a {destino}")
    hanoi(n - 1, auxiliar, destino, origen)

hanoi(3, "A", "C", "B")
```